

Sprint 2 Demo Script — Dartly

Date: April 21, 2026 **Slot:** 15 minutes total (8 min Phase A + 7 min Phase B)

Each section has three parts:

- **Say:** the exact line to deliver
 - **Show:** what clicks / taps happen in the browser
 - **Prove:** the file/line to open in the editor when asked for evidence
 - **Why:** the decision behind what's on screen, ready for a follow-up question
-

Pre-Demo Checklist

- ☐ App running at deployed URL or `localhost:3000`
 - ☐ Logged-out state (start fresh)
 - ☐ Seed data loaded:
 - **User A** (`testuser@gmail.com` / `Testpass1!`) — 8+ jobs across every stage, 2+ experiences, 2+ educations, 2+ skills, career prefs filled in, 2+ AI-generated documents linked to a job
 - **User B** (`testuser2@gmail.com` / `Testpass1!`) — ownership denial demo account
 - ☐ CI tab open on the latest green main-branch run
 - ☐ Terminal with `git log --oneline -15` ready (merged S2 PRs visible)
 - ☐ Credentials + one noted job ID in a scratch file
 - ☐ `bun run test` passes locally (133 tests)
-

Phase A — Scripted Product Demo (8 minutes)

A1. Dashboard Search / Filter / Sort (2 min)

Say:

"Sprint 2 turned the dashboard from a static list into a real application board. We have full-text search, stage filtering, and multi-axis sort — all running client-side over the user's full job set."

Show:

1. Dashboard loads — stage badges on every card, active count in the header.
2. Type **"Acme"** in the search box → cards filter instantly across title, company, location, description, and custom notes.
3. Clear the search, open the filter dropdown → select **Interview** → list narrows.

4. Switch sort from **Recent** to **Company (A→Z)** → cards reorder.
5. Point at the color-coded stage badges: Interested / Applied / Interview / Offer / Rejected / Archived.

Prove:

- `src/utils/search-jobs.ts` — `searchJobs()` is the client-side filter. It's case-insensitive and fans across title, company, location, description, and customNotes.
- `src/tests/utils/search-jobs.test.ts` — 9 tests covering empty query, partial matches, case insensitivity, and field fan-out.

Why:

"Server returns all of the user's jobs on load. Search, filter, and sort run in the browser because there are at most a few hundred rows per user — pagination would cost us more in UX friction than it saves in bytes. If a power user ever hits 10k jobs, we move the same predicate to the server without changing the UI contract."

A2. Job Detail Workflow (2 min)

Say:

"Any card expands into a full job detail view. The stage picker, deadline, recruiter contact, and custom notes all persist through the same controlled API."

Show:

1. Click any card → detail view opens with the Overview section.
2. Point out editable fields: title, company, location, compensation, deadline, recruiter/contact notes, custom notes.
3. Edit the deadline inline → save → toast confirms.
4. Open the stage picker → move from **Applied** → **Interview**.
5. Overview badge updates, the "last activity" indicator refreshes.
6. Refresh the page (F5) — new stage persists.

Prove:

- `src/services/jobs.ts:74-108` — `updateJob()`. The whole update runs in a `prisma.$transaction`. Inside, when `stageChanged` is true, a `JobStageHistory` row is written in the same transaction as the job row. Either both land or neither does.
- `src/services/jobs.ts:49-72` — `createJob()` also writes an initial `JobStageHistory` row with `fromStage: null`, so every job has a complete history from creation.

Why:

"Stage history is the source of truth for the timeline and the response-rate metric. Writing it in the same transaction as the stage change means the timeline can never drift from the current stage —

there's no cache to invalidate, no reconciliation job. It also lets us treat history as append-only audit data, which simplifies the metrics math downstream."

A3. Interview / Activity / Outcome (2 min)

Say:

"The detail view has a single activity timeline that merges stage changes, interviews, follow-ups, and outcomes into one chronological feed."

Show:

1. Scroll to the **Activity Timeline** — stage changes already show.
2. Click **Add Interview** → modal for round type (Phone / Technical / Onsite), scheduled time, notes.
3. Save → the interview appears on the timeline with a round-type badge.
4. Click **Add Follow-Up** → modal for title + reminder date → save → appears on the timeline with a completion checkbox.
5. Open the outcome control in the header → pick **Offered** → badge updates, `outcomeAt` is persisted.

Prove:

- `prisma/schema.prisma` → `JobActivity` model. Note the `type` field is a discriminator string (Interview, FollowUp, etc.), and `scheduledAt`, `roundType`, `completed` are optional fields.
- `prisma/schema.prisma` → `JobStageHistory` model → `fromStage`, `toStage`, `changedAt`. This is what the timeline reads from for stage events.
- `prisma/schema.prisma` → `Job.outcomeAt` column.

Why:

"One table with a `type` discriminator instead of four separate tables for interview / follow-up / call / email. It means one set of queries drives the whole timeline, one set of RLS policies, and adding a new activity type is a string constant — not a migration."

A4. Profile + AI Document Generation + Metrics (2 min)

Say:

"The profile feeds AI document generation; the resulting documents are versioned and linked to the job; the metrics panel reads from the same stage-history data the timeline uses."

Show:

1. Click **Profile** → experience, education, skills, and career preferences all filled in.
2. Click **Edit** on an experience → modal with date picker, "currently working here" toggle, description field. Save.

3. Drag to reorder experiences → order persists and is used when building the resume.
4. Back to dashboard → open a seeded job → **Documents** tab → existing linked documents visible.
5. Click **Generate Resume** → resume draft appears, rendered from Markdown, saved as version 1, linked to this job.
6. Click **Generate Cover Letter** → same flow, different template.
7. Open a draft → click **Improve: Make Concise** → before/after diff.
8. Back to dashboard → point at the **Metrics Panel**: stage counts, response rate %, average days to response, active-application count.

Prove:

- `src/services/ai.ts` — `generateResumeDraft`, `generateCoverLetterDraft`, `rewriteContent`. Every one emits `logger.warn("AI stub provider invoked...")` on call so logs make it obvious we're not hitting a real model yet.
- `src/services/documents.ts:51-84` — `createDocument()` — document + version 1 + `JobDocumentLink` all written in the same transaction.
- `src/app/api/ai/resume/route.ts:1-30` — route does `requireAuth()` → `checkRateLimit(10/min)` → `validateBody(GenerateDocumentSchema)` → fetch profile + job → call stub → save as document.
- `src/services/metrics.ts:12-99` — computes stage counts, response rate (fraction of post-INTERESTED jobs that have a `JobStageHistory` row with `fromStage: "APPLIED"`), average days to response.

Why:

"The AI provider is deliberately a stub today — deterministic Markdown templates — because picking an LLM provider is a product decision we didn't want to block Sprint 2 on. The return shape is what a real model would produce, so the swap later is a one-file change with zero changes to routes or callers. The stub also means our tests don't need a mocked LLM client, and CI doesn't need an API key."

Phase B — Technical Evidence and Q&A (7 minutes)

B1. Workflow & Data Integrity (2 min)

Say:

"Workflow integrity is enforced in the service layer, not the UI. Three specific guarantees: stage history is written in the same transaction as the stage change; document versions are serializable to prevent duplicate numbers under concurrent writes; and duplicate job-document links surface as a 409 instead of a 500."

Show & prove:

1. Open `src/services/jobs.ts:81-104` — the transaction wrapping `job.update` and the conditional `jobStageHistory.create` .

"If the stage changed, we write the history row inside the same transaction. Nothing else can observe the job in the new stage without a matching history row already committed."

2. Open `src/services/documents.ts:40-49` — the `nextVersionNumber` helper using `aggregate({ _max: versionNumber })` .
3. Open `src/services/documents.ts:110-132` — `updateDocumentContent` wraps the whole read-modify-write in `prisma.$transaction` with `{ isolationLevel: "Serializable" }` .

"This was a real race. Before Sprint 2 hardening, two concurrent saves could both read the same max version number and both write version N+1 — duplicate rows with no DB uniqueness to stop them. We fixed it at the application level because the shared dev database has drift that blocks a migration mid-sprint; the serializable transaction is the strongest guarantee Postgres gives us without schema changes."

4. Open `src/services/documents.ts:193-207` — the P2002 catch on `jobDocumentLink.create` .

"Linking a document version to a job a second time violates `@@unique([jobId, documentVersionId])` . We translate that into a clean `ApiError(409, "Document version already linked to this job")` so the client sees a meaningful error code, not a stack trace."

5. Open `src/tests/services/documents.test.ts` → the **"computes next version number from max() inside transaction"** block.

"The test asserts both the isolation level and the aggregate shape. If someone refactors and drops the serializable wrapper, CI fails on the next push."

Why this set of choices:

"Data integrity at the service layer, not the UI, because the UI is the untrusted caller. Even a buggy client, a direct curl, or a future background job hits the same guarantees. And we pushed the fix into application code rather than schema because our dev DB is shared and a migration mid-sprint would wipe teammate data — the trade-off is we'll land the matching `@@unique` constraint as a coordinated Sprint 3 migration."

B2. CI and Unit Testing Evidence (3 min)

Say:

"CI enforces lint, type-check, tests, and build on every PR. Sprint 2 took us from 60 tests to 133, with explicit negative-path coverage across every new surface."

Show & prove:

1. Open GitHub → Actions tab → latest green run on `main` .

"Green checks: install, prisma generate, lint, type-check, test, build."

2. Open `.github/workflows/ci.yml` .

"This runs on every PR and every push to main. No way to merge a red branch."

3. Open `src/tests/services/metrics.test.ts` .

"Dashboard logic test. Given 5 jobs past INTERESTED and 3 `JobStageHistory` rows showing `APPLIED` → `INTERVIEW` , `responseRate` is 60%. This is the exact metric the demo panel shows."

4. Open `src/tests/services/documents.test.ts` → the **"computes next version number from max() inside transaction"** block.

"Non-happy-path test for the version race — asserts the isolation level, the aggregate call, and the computed version number."

5. Open `src/tests/api/ai-resume.test.ts` → the **"returns 400 when request body validation fails"** and **"returns 429 when rate-limited"** blocks.

"Two negative paths on a single route: malformed body → 400, rate limit hit → 429. Both return the API's consistent `{ error: string }` shape."

6. Open `src/tests/api/jobs-id.test.ts` → the ownership denial test.

"If User A tries to access User B's job, we return 404 — not 403. An attacker can't even tell if the resource exists."

7. Run `bun run test` in a terminal.

"133 tests, all passing, under ten seconds."

Why these tests specifically:

"Sprint 2's rubric explicitly penalizes happy-path-only tests. We added 14 new tests this week targeting the failure modes that are easy to regress: auth bypass, ownership leak, validation leak, rate-limit bypass, and race conditions on write. Every negative-path test is a tripwire for a future refactor that removes a safety check."

B3. Architecture and Implementation Q&A (2 min)

Prepared answers. Whoever knows the area best takes the question.

Q: How does Sprint 2 extend Sprint 1 without breaking prior flows?

"Every new feature lives in new tables or new nullable columns — `JobStageHistory` , `JobActivity` , `Document` , `DocumentVersion` , `JobDocumentLink` . The Sprint 1 `Job` table gained `deadline` , `recruiterNotes` , `customNotes` , `outcomeAt` — all nullable, no data migration required. Every Sprint 1 test still passes unchanged."

Pointer: `prisma/schema.prisma` shows the new tables and the nullable columns.

Q: Where does stage-transition logic live, and why?

"In `src/services/jobs.ts` , inside `updateJob` . We kept it in the service layer so the API route stays thin and the same function can be called by a CLI, a background job, or a future webhook — every caller gets the same history-write guarantee."

Q: How do timeline, interviews, and documents coordinate inside Job Detail?

"Timeline is a projection over `JobStageHistory` + `JobActivity` ordered by timestamp. Interviews and follow-ups are both rows in `JobActivity` with a `type` discriminator — one table, one query. Documents join through `JobDocumentLink` which points at a specific `DocumentVersion` , so we can pin exactly which draft was sent."

Q: What profile entities feed the AI, and how?

"`getProfile(userId)` returns full `ProfileData` — experiences, educations, skills, summary, target roles. Combined with `Job { title, company, description }` into a `JobContext` , both passed into the generator functions in `src/services/ai.ts` . The generator is swappable; today it's a deterministic template, tomorrow it's a provider call with the same signature."

Q: What technical debt did Sprint 2 introduce, and how will Sprint 3 pay it down?

"Three items. One: AI provider is a stub — Sprint 3 has an `AI_PROVIDER` env gate and provider wiring planned. Two: `DocumentVersion` is protected at the application layer with a serializable transaction but has no DB-level `@@unique([documentId, versionNumber])` yet — that's a coordinated migration we're landing before deploy. Three: the dashboard still loads full job rows when only `id` and `stage` are needed for badges; metrics already uses `select` to trim that and the rest of the dashboard will follow."

Timing Guide

Clock	Section
0:00	A1 — Search / Filter / Sort
2:00	A2 — Job Detail
4:00	A3 — Interview / Activity / Outcome
6:00	A4 — Profile + AI + Metrics
8:00	Phase B starts
8:00	B1 — Workflow integrity
10:00	B2 — CI & Tests
13:00	B3 — Architecture Q&A
15:00	Done

Emergency Fallbacks

Problem	Fallback
App won't load	Screenshots of every A-section saved locally; show code and walk through the flow
AI generation errors	Use pre-seeded documents; explain the route flow on the code side
Job creation/edit fails	Show the API route + Prisma schema; point at the green CI
CI tab unavailable	Show <code>.github/workflows/ci.yml</code> + local <code>bun run test</code> output
Seed data missing	<code>bun run db:seed</code> (15s) or create one job live
Rate limit hits mid-demo	That is the B1 evidence — the rate limit is real. Wait 60s.

Files to Have Open (Quick Reference)

Evidence	File	Anchor
Search logic	<code>src/utils/search-jobs.ts</code>	full file
Stage transition + history write	<code>src/services/jobs.ts</code>	<code>updateJob</code> (lines 74-108)
Initial history on create	<code>src/services/jobs.ts</code>	<code>createJob</code> (lines 49-72)
Version race fix (serializable tx)	<code>src/services/documents.ts</code>	<code>updateDocumentContent</code> (lines 110-132) + <code>nextVersionNumber</code> helper (40-49)
Duplicate link 409	<code>src/services/documents.ts</code>	<code>linkDocumentToJob</code> P2002 catch (lines 193-207)
Rate-limited AI route	<code>src/app/api/ai/resume/route.ts</code>	lines 1-30
Stub provider warn	<code>src/services/ai.ts</code>	<code>warnStub</code> helper
Metrics math	<code>src/services/metrics.ts</code>	full file
Version race test	<code>src/tests/services/documents.test.ts</code>	"computes next version" block
Metrics unit test	<code>src/tests/services/metrics.test.ts</code>	full file
Profile completion test	<code>src/tests/services/profile.test.ts</code>	<code>computeCompletion</code> block
AI negative path	<code>src/tests/api/ai-resume.test.ts</code>	"returns 400" + "returns 429" blocks
Ownership denial	<code>src/tests/api/jobs-id.test.ts</code>	ownership block
Auth guard	<code>src/lib/requireAuth.ts</code>	full file
Rate-limit helper	<code>src/lib/rate-limit.ts</code>	full file
CI config	<code>.github/workflows/ci.yml</code>	full file
Schema overview	<code>prisma/schema.prisma</code>	full file